MCU: Exact and Constant-Round Nonlinear Function Evaluation in MPC without Preprocessing

Min Yang, Jinxuan Du, Zihang Zhou, Dongcan Guo, and Qingshu Meng

Wuhan University, Hubei Province 430072, China qsmeng@126.com

Abstract. The rapid proliferation of artificial intelligence (AI) applications—from large language models to deep neural networks—has led to an unprecedented demand for massive training data. This, in turn, has intensified privacy concerns, as sensitive information may be exposed during collaborative model training and inference. Privacy-preserving computing, particularly secure multi-party computation (MPC), offers a rigorous technical solution. Since Yao's garbled circuits, nearly four decades of research have brought significant progress, yet MPC still faces fundamental challenges in efficiency and scalability. A handful of efficient and scalable MPC systems exist, but they almost invariably require heavy offline preprocessing—a burden that becomes prohibitive for modern Transformer-based architectures with hundreds of billions of parameters. Conversely, the few approaches that eliminate preprocessing cover only a limited set of nonlinear functions (e.g., comparisons and ReLU), leaving a critical gap for real-world machine learning.

In this paper, we propose MCU (Mask-Compute-Unmask), a novel MPC architecture that directly supports a broad spectrum of nonlinear functions without any offline preprocessing. MCU introduces a semi-honest, non-colluding helper party (HP) that acts as an active computational engine: parties additively mask their inputs, the HP computes the exact target function on the aggregated masked data, and the parties then unmask using locally known masks. This simple yet powerful paradigm enables a suite of constant-round, scalable protocols, including general multiplication (2 rounds), power functions (2 rounds), exponentials and trigonometric functions (4 rounds), and sigmoid and softmax (6 rounds). All protocols achieve simulation-based security and rely solely on additive masks and synchronized pseudorandom generators—no preprocessing, no polynomial approximations, and no accuracy loss. Unlike prior preprocessing-free works that are confined to comparison-related functions (e.g., Bicoptor [32]), MCU supports a full spectrum of nonlinear functions essential for modern DNNs and Transformers. MCU effectively overcomes long-standing efficiency and scalability barriers in privacy-preserving machine learning, offering a practical solution for deploying large models such as Transformers in privacy-sensitive environments.

1 Introduction

1.1 The AI Privacy Dilemma

The advent of large language models (LLMs) and deep neural networks (DNNs) has transformed artificial intelligence, enabling unprecedented capabilities in natural language processing, computer vision, and beyond. However, this progress comes at a steep price: the hunger for data. Training state-of-the-art models requires aggregating vast amounts of potentially sensitive information—personal conversations, medical records, financial transactions, and proprietary business data. When multiple institutions wish to collaborate on model training or inference without revealing their private data, the need for privacy-preserving computation becomes acute.

Privacy-preserving computing, and specifically secure multi-party computation (MPC) [2,8,27], provides a theoretically sound answer. Since Yao introduced garbled circuits in the 1980s[27], the field has witnessed remarkable advances. Today, MPC protocols can support various machine learning tasks with provable security guarantees. Yet, despite nearly forty years of development, two fundamental challenges remain: **efficiency** and **scalability**.

1.2 The Preprocessing Bottleneck

Recent years, a number of efficient MPC systems have emerged, particularly those based on secret sharing. Many of them achieve impressive online performance by moving most of the heavy cryptographic work to an offline **preprocessing** phase [6,16,5,18,13]. In this paradigm, parties generate correlated randomness (e.g., Beaver triples) before seeing the actual inputs, and the online phase becomes lightweight.

Unfortunately, preprocessing introduces its own set of problems. First, it requires storage and communication of large amounts of random data—for a Transformer model with billions of parameters, the preprocessing overhead can be astronomical. Second, the preprocessing phase itself is an MPC problem, often requiring either a trusted dealer or additional rounds of interaction. To mitigate this, many state-of-the-art systems introduce a semi-honest, non-colluding helper party (HP) solely for offline randomness generation [22,24,13,12,25,20]. While this reduces the burden on the main parties, it does not eliminate preprocessing; it merely shifts the problem to a different entity.

The deployment cost becomes prohibitive when models grow to hundreds of billions of parameters, as is typical for LLMs. Moreover, the preprocessing overhead scales with the number of nonlinear operations, which in Transformers include GeLU, softmax, and exponentials—functions that are notoriously hard to evaluate efficiently in MPC.

1.3 The Nonlinear Function Gap

A handful of recent works have attempted to eliminate preprocessing altogether by using synchronized pseudorandom generators (PRGs) to generate masks on

the fly [29,32]. These approaches are attractive because they require no offline storage. However, they cover only a very limited set of nonlinear functions—mainly comparisons, ReLU, and simple arithmetic. In particular, Bicoptor [32] is restricted to comparison-related operations, and cannot handle exponentials, trigonometric functions, sigmoid, or softmax. For the rich nonlinearities that power modern DNNs and LLMs (exponential, sigmoid, softmax, GeLU, trigonometric functions), most existing schemes [15,19,7,26,31,28,11,23,9,4,3] require preprocessing. Among them, some resort to polynomial approximations or iterative methods [15,19,7,26,31,11,23], which reintroduce accuracy loss and increase communication rounds. Others rely on M2A protocols [28,3], introducing $\log_2 n$ rounds of communication, or on function secret sharing (FSS) [9,4], which demands substantial offline preprocessing and storage.

1.4 The MCU Paradigm: Mask, Compute, Unmask

In this paper, we propose **MCU** (Mask-Compute-Unmask), a new MPC architecture that directly supports a wide range of nonlinear functions **without any offline preprocessing** and **without polynomial approximations**. The core idea is a fundamental redefinition of the helper party's role: instead of a passive source of random masks, the HP becomes an **active computational engine**.

The workflow is deceptively simple:

1. **Mask**: Each party additively masks its input using synchronized randomness (generated on-the-fly from a shared PRG) and sends the masked value to the HP.
2. **Compute**: The HP aggregates the masked values, computes the **exact** target function on the aggregate, and redistributes additive secret shares of the result to the parties.
3. **Unmask**: Each party uses its local knowledge of the masks to recover its final share of the function output.

The apparent simplicity belies a deep technical challenge: for each function f, we must find an algebraic identity that allows parties to recover $f(x)$ from $f(x+r)$ using only the locally known mask r. For many functions of practical interest—multiplication, power, exponential, trigonometric, sigmoid, softmax—we have identified such identities and designed corresponding constant-round protocols.

What makes MCU different from prior direct-computation approaches. Prior works such as Bicoptor [32] are confined to comparison-related functions. In contrast, MCU provides the first constant-round, preprocessing-free protocols for exponentials, trigonometric functions, sigmoid, and softmax—functions that are indispensable for modern Transformer models. Moreover, MCU naturally scales to any number of parties, whereas Bicoptor is designed exclusively for three parties.

1.5 Our Contributions

We present the MCU framework and a comprehensive protocol suite with the following characteristics:

- **No offline preprocessing**: All randomness is generated online via synchronized PRGs. Parties only exchange a one-time seed before any computation; no per-circuit correlated randomness is stored or communicated.
- **Exact computation**: For algebraic functions, results are exact over the ring $\mathbb{Z}_{2^l}$. For transcendental functions, we use standard floating-point arithmetic with no additional approximation beyond the inherent rounding of the math library.
- **Constant communication rounds (independent of the number of parties)**:
  - General multiplication and power x^k: **2 rounds**
  - Exponential a^x and trigonometric functions: **4 rounds**
  - Sigmoid and softmax: **6 rounds**

  These round counts hold for *any* number of parties n. In contrast, previous preprocessing-free frameworks either support only comparison-related functions (e.g., Bicoptor [32]) or introduce additional rounds when scaling to multi-party settings.
- **Broad function coverage**: MCU handles multiplication, power, exponentials, trigonometric functions, sigmoid, and softmax—covering the essential nonlinearities in modern DNNs and Transformers.
- **Scalability**: The HP bears most of the computational burden, and communication grows only linearly with the number of parties. Our multiplication protocol scales better than Beaver-based multiplication in multi-party settings (see Section 5).
- **Provable security**: All protocols are proven secure in the semi-honest model under the assumption that the HP does not collude with any party. The proofs follow the simulation paradigm.

To the best of our knowledge, MCU is the first MPC framework that simultaneously achieves **preprocessing-free**, **constant-round**, and **exact** evaluation of such a broad class of nonlinear functions—including exponentials, trigonometric functions, sigmoid, and softmax, none of which were supported by prior preprocessing-free works.

1.6 Why This Matters Now

The rise of LLMs has made efficient privacy-preserving inference an urgent industrial need. Cloud-based LLM services must protect user prompts and model weights simultaneously. Current MPC solutions either suffer from prohibitive preprocessing (making them impractical for large models) or rely on polynomial approximations that sacrifice accuracy and round efficiency. MCU breaks this deadlock by enabling exact, constant-round evaluation of all key nonlinearities without any preprocessing. Its multiplication protocol can also accelerate existing schemes that rely on multiplicative masks or the M2A protocol [28,3,17].

1.7 Paper Organization

The remainder of this paper is organized as follows. Section 2 defines the MCU architecture, security model, and computational model. Section 3 presents the full suite of protocols. Section 4 provides security proofs. Section 5 reports experimental results. Section 6 discusses related work and addresses potential concerns. Section 7 concludes.

2 The MCU Architecture

2.1 Security Model and its rationale

We consider the semi-honest (honest-but-curious) adversary model, in which parties follow the protocol specifications but may attempt to infer additional information from their views. The HP is assumed to be non-colluding. We argue that this assumption is rational from two aspects.

1. A cheap way to address the preprocess problem in standard MPC setting. In the standard end-to-end MPC, even the simplest multiplication operation requires offline pre-generation of Beaver triples to eliminate the randomness introduced in additive masks. For other complex nonlinear functions, if solutions exist, they also necessarily require offline precomputation, which implies great deployment cost, especially for DNN with large number of parameters. What's more, the problem to be solved in offline-phase itself is also an MPC problem. To solve the MPC problem in the offline phase as efficiently and early as possible, many efficient solutions introduce a semi-honest, non-colluding third-party server [22,24,13,12,25,20] or cryptographic commodity server[1,30]. That is, introducing a non-colluding HP to achieve efficiency gains and security guarantees is an accepted and cheap trade-off in the community.

2. Crucially, this assumption is grounded in realistic deployment scenarios. Our real world is dominated by all kinds of centralized organizations. Examples include:

- **Financial dark pools**: A securities regulator (e.g., the SEC) operates the trading platform, matching buy and sell orders without colluding with any participating institution.
- **Healthcare data sharing**: A health authority (e.g., a national health service) aggregates medical records from hospitals for research or statistics, and is inherently motivated to protect patient privacy and maintain impartiality.
- **Industry consortia**: A trade association collects and analyzes member companies' operational data (e.g., supply chain efficiency) to provide benchmarking services; the association has no incentive to favor any particular member.

In such scenarios, the HP—acting as a regulator, authority, or association—naturally fulfills the role of a fair, non-colluding, semi-honest party, while the regulated parties may have incentives to collude for unfair benefits. MCU is designed precisely for these scenarios.

2.2 MCU Components

The MCU framework relies on two types of pseudorandom generators (PRGs) to achieve communication-free randomness generation:

- **PRG**: A cryptographically secure pseudorandom generator shared among all participating servers $P_1, \ldots, P_n$ with identical seeds. This enables parties to generate the same sequence of random numbers without any communication.
- **ASPRG**: A unique pseudorandom generator shared between the HP and each individual party P_i, each instantiated with distinct seeds. This allows the HP to privately distribute shares to each party.

Initialization. Before protocol execution, a one-time seed establishment is performed for both the PRG and the ASPRGs. This setup is lightweight and does not involve per-circuit correlated randomness (e.g., Beaver triples), distinguishing MCU from preprocessing-heavy MPC systems.

2.3 Operational Flow

The MCU framework computes a function $f(x_1, \ldots, x_n)$, where x_i is the private input of party P_i, and outputs additive secret shares of the result to all parties. The protocol proceeds in three steps:

1. **Mask**: Each party P_i uses PRG_0 to generate a random mask r_i. It computes $m_i = x_i + r_i \mod 2^l$ (additive masking) and sends m_i to the HP.
2. **Compute**: The HP aggregates the received values to obtain $M = \sum_{i=1}^{n} m_i = x + r \mod 2^l$, where $x = \sum x_i$ and $r = \sum r_i$. It then computes the desired value $v = g(M)$, where g is a function derived from f (e.g., if $f = a^x$, then $g(M) = a^M$). Next, the HP generates random shares $s_1, \ldots, s_{n-1}$ using the ASPRG, sets $s_n = v - \sum_{i=1}^{n-1} s_i$, and sends s_n to P_n.
3. **Unmask**: Each party P_i obtains its share s_i (for $i < n$, s_i is generated locally via ASPRG_i; for $i = n$, it is received from the HP). Using the local knowledge of r (reconstructed from the r_i via PRG_0), each party computes its final share $\langle f(x) \rangle_i = h(s_i, r)$, where h depends on the specific function being evaluated.

The key observation is that the HP only sees $M = x + r \mod 2^l$ with r uniformly random, and thus learns nothing about x. This property forms the foundation of provable security for all additive-mask protocols in the MCU framework.

2.4 Computational Model

Our protocols operate in two distinct computational settings depending on the function type.

Algebraic Functions. For functions such as multiplication and power, computation is performed over a ring $\mathbb{Z}_{2^l}$. All operations—addition, multiplication, and reconstruction—are exact and deterministic. The protocols produce shares

that reconstruct to the exact value (only with random rounding error) that would be obtained by evaluating the function in the same algebraic structure.

Transcendental Functions. For functions such as exp, sin, cos, sigmoid, and softmax, each input is masked $x_i+r_i \mod 2^l$. The HP reconstructs a masked value $x+r \mod 2^l$ first and then turns it into a floating-point number. The evaluation of the function is performed over floating-point numbers using standard arithmetic (e.g., IEEE 754). We assume that all parties have access to a deterministic floating-point implementation of these functions (e.g., from a standard math library). The final reconstructed output is identical to the result of evaluating the function on the plaintext input (only with random rounding error) using the same floating-point arithmetic. No additional approximation is introduced by the MPC protocol.

3 Provably Secure Protocols

We now present the protocols. For each, we highlight the unmasking challenge and describe how we address it.

3.1 General Multiplication

Suppose $x_1, x_2, \cdots, x_k$ are k variables. By inclusion-exclusion,

$$\prod_{j=1}^{k} x_j = \prod_{j=1}^{k}(x_j + r_j - r_j) = \sum_{S\subseteq\{1,\ldots,k\}} (-1)^{|S|} \left(\prod_{j\in S} r_j\right)\left(\prod_{j\notin S}(x_j + r_j)\right). \quad (1)$$

Protocol 1. Multiplication Protocol

Input: Each P_i holds $\langle x_1\rangle_i, \ldots, \langle x_k\rangle_i$ with $x_j = \sum_i \langle x_j\rangle_i$.

Output: Each P_i obtains $\langle \prod_{j=1}^{k} x_j\rangle_i$.

1. Each party P_i (in parallel) compute $a_{j,i} = \langle x_j\rangle_i + \langle r_j\rangle_i$ for $j = 1, \ldots, k$. Send k values $a_{1,i}, \ldots, a_{k,i}$ to HP.
2. HP reconstructs $\sum_{m=1}^{n} a_{j,m} = x_j + r_j$, $j = 1, \ldots, k$. For each $S \in \{1, 2, \cdots, k\}$, HP calculates $A_S = \left(\prod_{j\notin S}(x_j + r_j)\right)$, generates shares $t_{S,1}, \ldots, t_{S,n-1}$ via $ASPRG_1, \cdots, ASPRG_{n-1}$ respectively, sets $t_{S,n} = A_S - \sum_{j=1}^{n-1} t_{S,j}$, and sends $t_{S,n}$ to P_n. There are 2^k different S, as the full set S is unnecessary to share in ASS way.
3. Each party P_i (in parallel) compute $\langle \prod_{j=1}^{k} x_j\rangle_i = \sum_{S\subseteq\{1,\ldots,k\}} (-1)^{|S|} \left(\prod_{j\in S} r_j\right) t_{S,i}$ as its output share. P_1 calculate $\langle \prod_{j=1}^{k} x_j\rangle_1 = \langle \prod_{j=1}^{k} x_j\rangle_1 + (-1)^k r_1 r_2 \cdots r_k$.

Unmasking Challenge: Intuitively to obtain $\langle \prod_{j=1}^{k} x_j\rangle_i$, we should subtract all items with degree less than k from $\prod_{j=1}^{n}(x_j + r_j)$. That is a little complex. By formula (1), it is easy.

Properties: The communication rounds of the protocol is only 2 at the cost that the protocol needs to transfer $2^k - 1$ ring elements to P_n. It is practical for k not big, say $k \leq 5$.

The protocol has three special cases.

- First Case: When $k = 2$, the problem the protocol solves becomes the classic 2-party multiplication problem.
- Second Case: If each variable x_j is specially shared that only one share is non-zero, the problem the protocol solves becomes the problem of M2A (transforming the multiplicative shares into additive shares).
- Third Case: If all k variables are equal to each other, the protocol becomes a power protocol. In this case, as the steps can be greatly simplified, we list it as an individual protocol.

3.2 Power Function x^k

Protocol 2. Power Protocol

Input: Shares of x.

Output: Shares of x^k.

1. **Mask**: Each P_i uses PRG_0 to generate a random r_i and sends $m_i = x_i + r_i$ to the HP.
2. **Compute**: The HP computes $R = \sum m_i = x + r$, where $r = \sum r_i$. For each exponent $i = 2, \ldots, k$, the HP computes $p_i = R^i$. For each i, the HP generates shares $s_{i,1}, \ldots, s_{i,n-1}$ via ASPRG, sets $s_{i,n} = p_i - \sum_{j=1}^{n-1} s_{i,j}$, and sends $s_{i,n}$ to P_n.
3. **Unmask**: Parties compute $r = \sum r_i$ via PRG_0. By the formula, $x^k = (x + r - r)^k = \sum_{j=0}^{k} \binom{k}{j} (x + r)^j (-r)^{k-j}$, each party P_i computes $\langle x^k \rangle_i = \sum_{j=1}^{k} \binom{k}{j} (-r)^{k-j} s_{j,i}$.

Unmasking Challenge: The Beaver-based approach requires $O(\log k)$ rounds via repeated squaring. To reduce the round complexity, the HP computes all $(x + r)^i$ in parallel, and the parties unmask simply.

Properties: The protocol requires two communication rounds, and is provably secure. The number of rounds is independent of the degree k, which is a significant advantage. Using the power protocol, many nonlinear functions, say the expensive division operation, can be evaluated in low constant rounds.

3.3 Sign function

The sign protocol is from the paper [21], which extends the 2-party scheme[32] to multi-party scheme. As we will use it for wrap protocol, we list it here. Its security lies on the two mask techniques. The share x_i of x are masked additively in an additive group and the shares $< v_i >$ of v_i are masked multiplicatively in a multiplicative group F_p, which reveal nothing about the hidden variables.

Sign(x) determines whether input $x \geq 0$. Returns 1 if $x \geq 0$, else 0.

Protocol 3. Sign Protocol

Input: $x = \sum_{i=1}^{n} x_i \mod 2^l$, each party P_i holds x_i, l_x is the bit-length of x and l, the modulus of the ring.

Output: Each party obtains sign(x).

1. Each party generates a random bit t using PRG_0 and computes $x_i = (-1)^t x_i$.
2. Each P_i uses PRG_0 to generate a random s_i and send $x_i + s_i$ to HP.
3. HP computes $x+s = \sum_{j=1}^{n}(x_j+s_j) \mod 2^l$. HP runs locally $\mathrm{trc}(x+s, j, l-l_x-j) \mod 2^{l_x}$ for $j = 0, \ldots, l_x-1$ in accordance with the rules in Bicopter 2.0[32], and sends each party one share $< u_j >=< \mathrm{trc}(x+s, j, l-l_x-j) > \mod 2^{l_x}$ in ASS way.
4. Each party calculates $s = \sum_{j=1}^{n} s_j \mod 2^l$ using PRG_0. Each party runs locally $\mathrm{trc}(-s, j, l-l_x-j) \mod 2^{l_x}$, and obtains one share $\langle w_j \rangle = \langle \mathrm{trc}(-s, j, l-l_x-j) \rangle \mod 2^{l_x}$. Calculates $\langle u_j \rangle = \langle u_j \rangle + \langle w_j \rangle \mod 2^{l_x}$, and $\langle v_j \rangle = \langle u_j + u_{j+1} - 1 \rangle \mod 2^{l_x}$ for $j = 0, \ldots, l_x - 1$, and $\langle v_{l_x} \rangle = \langle u_{l_x} - 1 \rangle \mod 2^{l_x}$.
5. Switch the modulus of $\langle v_j \rangle$ from $\mathbb{Z}_{2^{l_x}}$ to $\mathbb{Z}_p$ (still denoted $\langle v_j \rangle$) for $j = 0, \ldots, l_x - 1$.
6. Each party uses PRG_0 to generate the same permutation and shuffles $\Pi(< \{v_j\} >)$, still denoted as v_j for $j = 0, \ldots, l_x - 1$.
7. Each party generates $l_x + 1$ random numbers $\{r_j\}$ using PRG_0 for $j = 0, \ldots, l_x - 1$, where $r_j \in \mathbb{Z}_p^*$, $\mathbb{Z}_p^* := \mathbb{Z}_p \backslash \{0\}$. Masking: $< \{w_j\} >:=< \{v_j \cdot r_j\} >$ mod$p$ for $j = 0, \ldots, l_x - 1$, which hides v_j completely from HP for nonzero v_i.
8. Each party sends $< \{w_j\} >$ to HP for $j = 0, \ldots, l_x - 1$.
9. HP reconstructs $\{w_j\}$, and sets $sign(x) := 1$ if there exists 0 in $\{w_i\}$, otherwise $sign(x) = 0$.
10. HP broadcasts $sign(x)$ directly (or shares it in ASS way) to all parties.
11. Each party sets sign(x)=sign(x) if t=0; else sign(x)=1-sign(x) (in ASS way, <sign(x)> = sign(x) if t=0; else P_0 sets $< sign(x) >_0 = 1 - < sign(x) >_0$, $P_i, i > 1$ sets $< sign(x) >_i = - < sign(x) >_i$).

The sign protocol requires 4 rounds of communication.

3.4 Wrap Protocol

The objective is to determine whether $x + r > 2^l$. The wrap protocol returns 1 if true, else 0. Here $x = \sum_{i=1}^{n} x_i \mod M$ and $r = \sum_{i=1}^{n} r_i \mod M$ are shared among n parties, each holding x_i and r_i. $M = 2^l$.

From the identity $(x_i + r_i) = (x_i \mod M) + M \cdot \lfloor x_i/M \rfloor + (r_i \mod M) + M \cdot \lfloor r_i/M \rfloor$, we obtain

$$\left\lfloor \frac{\sum_{i=1}^{n}(x_i + r_i)}{M} \right\rfloor = \left\lfloor \frac{x+r}{M} \right\rfloor + \left\lfloor \frac{\sum_{i=1}^{n} x_i}{M} \right\rfloor + \left\lfloor \frac{\sum_{i=1}^{n} r_i}{M} \right\rfloor.$$

In MCU architecture, $\left\lfloor \frac{\sum_{i=1}^{n} r_i}{M} \right\rfloor$ can be calculated locally. HP can compute $\left\lfloor \frac{\sum_{i=1}^{n}(x_i+r_i)}{M} \right\rfloor$ from $x_i + r_i$, no modulo is taken. If $\left\lfloor \frac{\sum_{i=1}^{n} x_i}{M} \right\rfloor$ is found, then $\lfloor (x+s)/M \rfloor$ (i.e., the wrap value) can be derived.

Now we will use the sign protocol to calculate $\left\lfloor \frac{\sum_{i=1}^{n} x_i}{M} \right\rfloor$.

Protocol 4. Wrap Protocol

Input: Each party holds $x_i, r_i \in Z_M$ such that $x = \sum_{i=1}^{n} x_i, r = \sum_{i=1}^{n} r_i$, and l_x is the bit-length of x.

Output: Each party obtains wrap(x, r, M).

1. Each party sends $x_i + r_i$ without modulo operation to HP, and HP computes $\left\lfloor \frac{\sum_{i=1}^{n}(x_i+r_i)}{M} \right\rfloor$ and broadcasts it to all parties.
2. Each party computes $\left\lfloor \frac{\sum_{i=1}^{n} r_i}{M} \right\rfloor$ locally using PRG_0 without modulo operation.
3. For $j = 0, 1, \ldots, n-1$, select a larger ring with modulus G (say $G = M^2$) such that $\sum_{i=1}^{n}(x_i) < G$, and all parties run the sign protocol in ring Z_G with inputs $\{x_1 - j2^l + 1, x_2, \ldots, x_n\}$ to decide whether $x > j2^l$. The largest j for which the inequality holds is $\left\lfloor \frac{\sum_{i=1}^{n} x_i}{M} \right\rfloor$.
4. Using these three values, compute locally

$$\left\lfloor \frac{x+r}{M} \right\rfloor = \left\lfloor \frac{\sum_{i=1}^{n}(x_i+r_i)}{M} \right\rfloor - \left\lfloor \frac{\sum_{i=1}^{n} x_i}{M} \right\rfloor - \left\lfloor \frac{\sum_{i=1}^{n} r_i}{M} \right\rfloor$$

as the output of wrap$(x, r, 2^l)$.

We should notice that the output j does not reveal information about x modulo M. As step 1 and step 2 are independent from step 3, the wrap protocol requires 4 rounds of communication.

3.5 Exponential Function a^x

Protocol 5. Exponential Protocol

Input: Shares of x.

Output: Shares of a^x.

1. **Mask**: Each P_i sends $m_i = x_i + r_i$ to the HP.
2. **Compute**: The HP computes $R = \sum_{i=1}^{n}(x_i + r_i) = x + r \mod 2^l$ and $E = a^R$. It generates shares $s_1, \ldots, s_{n-1}$ via ASPRG, sets $s_n = E - \sum_{i=1}^{n-1} s_i$, and sends s_n to P_n.
3. **Wrap Detection:** All parties run the wrap detection protocol to calculate $w = wrap(x, r, 2^l)$.
4. **Unmask**: Parties compute $r = \sum r_i$. Each P_i sets $\langle a^x \rangle_i = s_i a^{w2^l - r}$.

Unmasking Challenge: The identity $a^{x+r} = a^x a^r$ is simple. If $x + r < 2^l$, $\langle a^x \rangle_i = s_i / a^r$. If $x + r \geq 2^l$, $\langle a^x \rangle_i = s_i a^{w2^l - r}$.
Properties: As the wrap detection is independent from the mask-compute-unmask process and is of 4 rounds of communication, the exponential protocol needs 4 rounds of communication. The protocol is provably secure.

3.6 Trigonometric Functions sin(x) and cos(x)

Protocol 6. Trigonometric Protocol
Input: Shares of x.
Output: Shares of $\sin(x)$, $\cos(x)$.

1. **Mask**: Each P_i sends $m_i = x_i + r_i$ to the HP.
2. **Compute**: The HP computes $R = \sum_{i=1}^{n}(x_i + r_i) = x + r$, then $S = \sin(R)$ and $C = \cos(R)$. It generates shares $s_1, \ldots, s_{n-1}$ and $t_1, \ldots, t_{n-1}$ via AS-PRG, sets $s_n = S - \sum_{i=1}^{n-1} s_i$, $t_n = C - \sum_{i=1}^{n-1} t_i$, and sends (s_n, t_n) to P_n.
3. **Wrap Detection:** All parties run the wrap detection protocol to calculate $w = wrap(x, r, 2^l)$.
4. **Unmask**: Parties compute $r = \sum r_i$. Using angle subtraction:

$$\langle \sin(x) \rangle_i = (s_i \cos(w2^l) + t_i \sin(w2^l)) \cos(r) - (t_i \cos(w2^l) - s_i \sin(w2^l)) \sin(r),$$

$$\langle \cos(x) \rangle_i = (t_i \cos(w2^l) - s_i \sin(w2^l)) \cos(r) + (s_i \cos(w2^l) + t_i \sin(w2^l)) \sin(r).$$

Unmasking Challenge: The core insight is to express $\sin(x)$ as $\sin((x+r)-r)$. Since $x+r$ is known to the HP (as R) and r is known locally to the parties, the sine difference formula yields a clean unmasking expression:

$$\sin(x) = \sin((x + r) - r) = \sin(x + r)\cos(r) - \cos(x + r)\sin(r).$$

A similar identity holds for $\cos(x)$.
Properties: As the wrap detection is independent from the mask-compute-unmask process, the protocol needs 4 rounds of communication. The protocol is provably secure.

3.7 Sigmoid Function

Definition: $\sigma(x) = \frac{e^x}{1+e^x}$.

Protocol 7. Sigmoid Protocol
Input: Shares of x.
Output: Shares of $\sigma(x)$.

1. **Exponential with random shift**: Parties run the exponential protocol to obtain shares of e^x.
2. **Denominator construction**: Each party uses PRG_0 to generate a random b and locally computes shares of e^b. It then computes $u_i = e^b (e^x)_i + \langle e^b \rangle_i$ and sends u_i to the HP.

3. **Compute fraction**: The HP aggregates $U = \sum u_i = e^b e^x + e^b$. It computes $F = \frac{e^{x+a}}{U}$ (note that e^{x+a} is known from Step 1). The HP redistributes shares of F using ASPRG.
4. **Unmask**: Each party receives a share v_i of F and computes $\langle \sigma(x) \rangle_i = \frac{v_i \cdot e^b}{e^a}$.

Unmasking Challenge: From Step 1, each party holds a secret share of $e^x + 1$. A direct approach would involve division, but by now MCU does not support a cheap division operation. To avoid expensive division, we make the denominator public to the HP. To hide the denominator, we introduce two independent random masks a and b and isolate them:

- The HP sees e^{x+a} (from Step 1) and $U = e^b e^x + e^b$ (from Step 3).
- Without knowledge of a and b, these two values are statistically independent: for any fixed e^x, e^{x+a} is uniformly random (over a), and U is uniformly random (over b).
- This "mask isolation" ensures that the HP's view is simulable.

Properties: The protocol requires 6 rounds, and is provably secure.

3.8 Softmax Function

Definition: $\text{softmax}(x_m) = \frac{e^{x_m}}{\sum_{j=1}^{k} e^{x_j}}$.

Protocol 8. Softmax Protocol
Input: Shares of x_i for $i = 1, 2, \ldots, k$.
Output: Shares of $\text{softmax}(x_m)$.

1. **Exponential for each** x_j: Parties run the exponential protocol (2 rounds) for each $j = 1, \ldots, k$ to obtain shares of e^{x_j}. The k exponentials can be executed in parallel.
2. **Sum**: Each party P_i locally computes $s_i = \sum_{j=1}^{k} (e^{x_j})_i$ and sends $e^t s_i$ to the HP, where t is a random number.
3. **Compute fraction**: The HP computes $S = \sum_{i=1}^{n} e^t s_i = \sum_{j=1}^{k} e^{x_j + t}$ and calculates $e^{x_m + r_m}/S$, where r_m is the mask used in step 1. It then generates and sends shares $v_i = \langle e^{x_m + r_m}/S \rangle_i$ in additive secret sharing form to all parties.
4. **Local Division**: Each party P_i computes $\langle \text{softmax}(x_m) \rangle_i = e^t v_i / e^{r_m}$.

Unmasking Challenge: From Step 1, each party holds a secret share of the denominator. To avoid expensive division, we make the denominator public. While it is impossible to extract the exact x_j from the denominator $\sum_{j=1}^{k} e^{x_j}$, it leaks information. To hide this information, we multiply the denominator by e^t.
Properties: The protocol requires 6 rounds of communication. The protocol is provably secure.

4 Security Proof

We prove the security of all protocols in the semi-honest model with a non-colluding HP. The proofs follow the simulation paradigm: for any adversary controlling a subset of parties, we construct a simulator that, given only the inputs of the corrupted parties and the outputs, produces a view indistinguishable from the real execution. As the security is discussed in the semi-honest setting, we can only consider the simulation of received data.

Assumptions:

- PRG_0 and ASPRG_i are cryptographically secure (their outputs are indistinguishable from random).
- The HP does not collude with any party.

For multiplication, x^k, $\sin(x)$ and e^x, in HP's view, the received $x_i + r_i$ can be simulated by random numbers and the aggregated value $x + r$ are simulable too. In the adversary's view who corrupted $t (t < n)$ parties, the only received data s_n (if party P_n is corrupted) is indistinguishable from random numbers because at least one share is unknown to the adversary.

The detailed proof for multiplication is as follows.

Theorem 1. *Given HP is semi-honest (no collusion), multiplication protocol is secure against a semi-honest adversary corrupting any t $(t < n)$ parties.*

Proof. 1. Under the semi-honest model, we only consider the received data of the colluding parties. If P_n is not corrupted, no simulation is needed. If P_n is corrupted, as the adversary neither knows the product A_S nor at least one party's share, the simulator can generate a random number r_n to simulate $t_{S,n}$. The rest of simulator's view can be generated in the same way as the real view. So the two views are indistinguishable.

2. No Inference of Honest Parties' Inputs: As all x_j and A_S are additively secret shared, every share is critical. Secret channels prevent eavesdropping on honest parties' communications with HP, the corrupted parties do not know $x_{i,j} + r_{i,j}$ of the non-colluding party P_j.

3. HP's View Simulation: The HP received data includes $\{x_{i,j} + r_{i,j} \mid i = 1, 2, \cdots, k, j = 1, 2, \ldots, n\}$. Since $r_{i,j}$ are random numbers unknown to HP, the Simulator can simulate this view by sampling randomly kn random numbers in ring Z_{2^l}, which is indistinguishable from the real view. As the aggregated values $x_i + r_i, i = 1, \ldots, k$ are additively masked data, no information can be inferred.

Theorem 2 (Sigmoid). *Given HP is semi-honest and non-colluding, the sigmoid protocol is secure against a semi-honest adversary corrupting any t $(t < n)$ parties.*

Proof. The protocol composes the exponential protocol and a second phase. The exponential part is secure. In the second phase, in HP's view, $u_i = e^b \langle e^x \rangle_i + \langle e^b \rangle_i$ is simulable since $\langle e^x \rangle_i$ and $\langle e^b \rangle_i$ are secret shares. From the aggregated $(e^{x+b} + e^b)$, HP cannot infer any information about x since b is uniform to HP. Because

a and b are random and independent, e^{x+a} and $(e^b e^x + e^b)$ are independent from each other, HP cannot infer any information about x from them too. In the adversary's view (who corrupts $t(t < n)$ parties), the view is indistinguishable from random numbers because there exists at least one unknown share. The wrap step is independent and secure.

Theorem 3 (Softmax). *Given HP is semi-honest and non-colluding, the softmax protocol is secure against a semi-honest adversary corrupting any t $(t < n)$ parties.*

Proof. The protocol composes the exponential protocol and a second phase. The calculation of exponentials is secure. The HP receives $s_i = e^r \sum_j \langle e^{x_j} \rangle_i$, which are uniformly random shares and simulable. From the sum $S = \sum_{j=1}^{k} e^{x_j + t}$, HP cannot infer any information about x_j due to the existence of random t. In the adversary's view who corrupted parties, the view is indistinguishable from random numbers because at least one share is unknown. The wrap step is independent and secure.

5 Experimental Evaluation

We conduct performance experiments to evaluate the performance of the proposed multiplication and precision experiments to evaluate the precision of exponential protocols. For multiplication, we compare against Beaver-based multiplication, which is a standard baseline for secret-sharing-based MPC. Additional experiments, including performance of sign, wrap, sigmoid, and softmax, as well as end-to-end inference on small Transformer models, are planned for the full version of this work.

5.1 Multiplication

We evaluate the scalability of the multiplication protocol (Protocol 1) with respect to the number of parties n. The runtime is measured as the total wall-clock time from start to completion, and we compare it against Beaver-based multiplication.

As shown in Figure 1, although MCU requires two communication rounds compared to the single round of Beaver-based multiplication, it achieves lower overall latency. This is because Beaver-based multiplication incurs significantly higher communication volume due to the need to distribute multiple correlated random shares among all pairs of parties. In contrast, our protocol sends only $2n + 1$ ring elements and leverages an active helper to perform the bulk of the computation.

The runtime of our multiplication protocol exhibits near-linear growth with a small slope, remaining consistently low as the number of parties increases. Beaver-based multiplication, however, shows a more rapid increase, becoming substantially slower in medium- to large-party settings. These results indicate that Protocol 1 scales better in multi-party environments and is less sensitive to the number of participants.

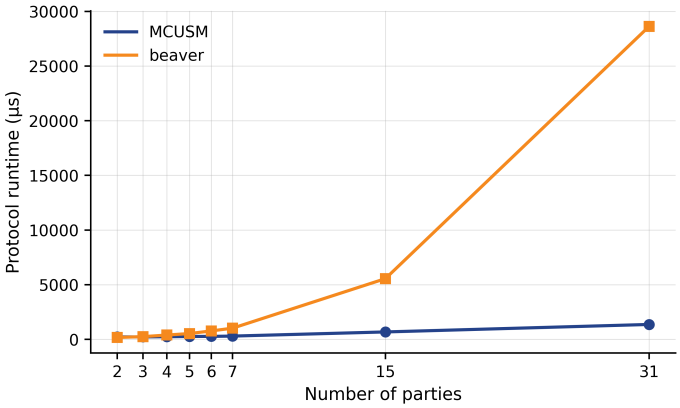

Fig. 1. Protocol runtime versus number of parties for Π_{MCUSM} and the Beaver-based method.

5.2 Precision of Exponential Protocol

We test the precision of the exponential protocol when x is near -20 or 20, which is almost the extreme value x can take in machine learning area. Our experiments show only rounding error exists.

6 Related Work and Discussion

6.1 Existing Approaches

Helper-assisted MPC. As summarized in Table 1, a number of works introduce a semi-honest helper party (HP) to improve efficiency [22,24,13,12,25,20,32,29]. In most of these systems [22,24,13,12,25,20], the HP is responsible for offline preprocessing (e.g., generating Beaver triples) and provides auxiliary assistance during the online phase. Wang et al. [25] construct several multiplication and exponentiation protocols, but their approach differs fundamentally from ours. In the work of Lu et al. [20], the HP participates as one of the parties in the online phase of a two-party computation protocol, serving a different role than in MCU.

In contrast, Zheng et al. [29] and Zhou et al. [32] take a more direct approach: they let the HP compute directly on masked data. Zheng et al. design protocols for ReLU, sigmoid, and tanh using random sign flipping and permutation as masking techniques. Permutation alone does not hide the numerical values; sorting the permuted inputs recovers the same sorted order as the original inputs, which can be used as a fingerprint for target recognition (e.g., via Euclidean-distance matching). Thus, permutation alone offers no provable security. Bicopter [32] focuses exclusively on comparison operations and ReLU; it does not support exponentials, trigonometric functions, sigmoid, or softmax—functions

that are essential for modern Transformer models. Despite these limitations, their insight—allowing the HP to compute on masked values—is highly inspiring. Building on this idea, we propose MCU, which systematically redefines the HP as an active computational engine that operates on additively masked data, enabling constant-round, exact evaluation of a broad class of nonlinear functions, including those not supported by prior preprocessing-free frameworks.

Table 1. Helper-assisted MPC works with non-colluding HP assumption

Work	Venue	HP Main Role
SecureML [22]	SP 2017	Triple generation
SecureNN [24]	PETS 2019	MSB computation
Asterisk [13]	SP 2024	Preprocessing & online
Asynchronous MPC [14]	ePrint 2025	Preprocessing & online
Wang et al. [25]	arXiv 2025	Preprocessing & online
Malicious MPC [20]	TDSC 2023	Preprocessing & online participant
Zheng et al. [29]	KBS 2022	Direct computation
Bicopter [32]	SP 2023	Direct computation

Non-linear functions in MPC. Recent years have seen growing interest in evaluating complex nonlinear functions [19,15,7,26,31,28,11] such as GeLU and softmax within MPC, driven by the need to protect data privacy in Transformer-based models. Most of these works rely on polynomial approximations or iterative methods, which inevitably introduce accuracy loss and require many communication rounds. In contrast, MCU evaluates these functions exactly (up to floating-point rounding) without polynomial approximations and without preprocessing.

6.2 Discussion

Potential concerns. A natural question regarding the MCU architecture is whether the introduction of an active HP introduces a single point of failure or a new trust assumption. We acknowledge that the non-colluding HP assumption is central to the security of MCU; however, this assumption is widely adopted in the MPC literature and aligns with many real-world deployment scenarios, such as regulated financial markets, healthcare data aggregators, and industry consortia, where a neutral third party naturally exists.

Another potential concern is the reliance on synchronized pseudorandom generators for mask generation. While this eliminates offline preprocessing, it requires that all parties share a common PRG seed. This is a lightweight setup that can be established once before any computation, and the use of cryptographic PRGs ensures security under standard assumptions.

Support for GeLU. Although we have omitted a dedicated GeLU protocol for brevity, the MCU framework can readily evaluate GeLU using its standard approximations (e.g., $x \cdot \text{sigmoid}(1.702x)$ or the tanh-based form [10]). Following the same additive masking approach, the HP computes the approximation on the masked input $x+r$, and the parties unmask using the local mask r. The only error incurred is the inherent floating-point rounding of the underlying math library; no additional MPC-induced error is introduced. Hence, MCU supports GeLU with the same constant-round and preprocessing-free properties as the other transcendental functions.

7 Conclusion

We introduced MCU, a novel MPC architecture that fundamentally redefines the role of a HP from a passive randomness source to an active computational engine. In the MCU framework, parties additively mask their inputs with synchronized randomness, the HP computes the target function directly on the aggregated masked data, and the parties then unmask the result using locally known masks. This approach enables scalable, low constant-round, and provably secure evaluation of nonlinear functions without any offline preprocessing or polynomial approximations.

We presented a suite of common protocols under the MCU framework, covering multiplication, power, comparison, wrap, exponential, trigonometric, sigmoid, and softmax functions. The multiplication and power protocols can speed up the evaluation of polynomials, which will benefit many approximation-based or M2A-based schemes.

While the core idea of MCU—mask, compute, unmask—may appear simple, its instantiation requires careful algebraic insights to derive unmasking identities for each target function, and for composite functions, sophisticated mask isolation techniques to prevent information leakage.

MCU offers a new paradigm for efficient and scalable privacy-preserving machine learning, particularly for large models such as Transformers, where nonlinear functions dominate the computational cost. We believe that the active-helper model opens up new avenues for research in helper-assisted MPC, and the algebraic unmasking techniques developed in this work may inspire further exploration of direct computation on masked data for other complex functions in standard MPC setting.

References

1. Donald Beaver. Commodity-based cryptography (extended abstract). In *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, STOC '97, page 446–455, 1997.
2. M. Ben-Or, S. Goldwasser, and A. Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC)*, 1988.

3. Yuntian Chen, Xianjia Meng, Zhiying Shi, Zhiyuan Ning, and Jingzhi Lin. Securetlm: Private inference for transformer-based large model with mpc. *Information Sciences*, 667:120429, 2024.
4. Ke Cheng, Yuheng Xia, Anxiao Song, Jiaxuan Fu, Wenjie Qu, Yulong Shen, and Jiaheng Zhang. Mosformer: Maliciously secure three-party inference framework for large transformers. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 4124–4138, New York, NY, USA, 2025. Association for Computing Machinery.
5. Ivan. Damgård and J. B. Nielsen. Scalable and unconditionally secure multiparty computation. In *Advances in Cryptology – CRYPTO 2007*, 2007.
6. Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Advances in Cryptology–CRYPTO 2012: 32nd Annual Cryptology Conference*, Lecture Notes in Computer Science. Springer, 2012.
7. Ye Dong, Wen-Jie Lu, Yancheng Zheng, et al. PUMA: Secure inference of LLaMA-7B in five minutes. *Security and Safety*, 4(2025014), 2025.
8. O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing (STOC)*, 1987.
9. Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: Secure gpt inference with function secret sharing. *Proc. Priv. Enhancing Technol.*, 2024:61–79, 2024.
10. D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus). arXiv:1606.08415, 2016.
11. Mazharul Islam et al. Compact: Approximating complex activation functions for secure computation. *Proceedings on Privacy Enhancing Technologies*, 2024:25–41, 2023.
12. Banashri Karmaka, Aniket Kate, and et al Shravani Patil. Breaking the barrier for asynchronous mpc with a friend. IACR ePrint Archive, Report 2025/???, 2025.
13. B. Karmakar et al. asterisk: Superfast MPC with a friend. In *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.
14. Banashri Karmakar, Aniket Kate, Shravani Patil, Arpita Patra, Sikhar Patranabis, Protik Paul, and Divya Ravi. Breaking the barrier for asynchronous mpc with a friend, 2025. `https://eprint.iacr.org/2025/1736`.
15. A. Y. L. Kei and H. S. M. Chow. SHAFT: Secure, handy, accurate, and fast transformer inference. In *Network and Distributed System Security Symposium (NDSS) 2025*, 2025.
16. M. Keller et al. MasCOT: faster malicious arithmetic secure computation with oblivious transfer. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
17. Zhongkai Li, Shuyang Fan, and Lingfei Jin. Protocol design of non-linear function in secure multi-party computation based on secret sharing. *Journal of Information Security and Applications*, 96:104293, 2026.
18. Fengrun Liu, Xiang Xie, and Yu Yu. Scalable multiparty computation protocols for machine learning in the honest-majority setting. In *USENIX Security Symposium*, 2024.
19. Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and Wenguang Chen. Bumblebee: Secure two-party inference framework for large transformers. In *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*, 2025.

20. Yibiao Lu, Bingsheng Zhang, and Kui Ren. Maliciously secure mpc from semi-honest 2pc in the server-aided model. *IEEE Transactions on Dependable and Secure Computing*, 21(4):3109–3125, 2024.
21. Zihang Zhou Jinxuan DU Min Yang, Dongcan Guo and Qingshu Meng. Esml: An efficient and scalable mpc scheme for privacy-preserving machine learning without precomputation, manuscript, 2026.
22. Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 19–38. IEEE, 2017.
23. Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4753–4771, 2024.
24. S. Wagh, D. Gupta, and N. Chandran. SecureNN: 3-party secure computation for neural network training. In *Proceedings on Privacy Enhancing Technologies (PETS) 2019*, 2019.
25. Kaiwen Wang, Xiaolin Chang, Junchao Fan, and Yuehan Dong. Efficient private inference based on helper-assisted malicious security dishonest majority mpc, 2025.
26. Shuya Yang, Xiaodong Li, and Jianyi Zhang. Privacy-preserving computing scheme for ciphertext neural network training. In *Proceedings of the 2024 3rd International Conference on Cryptography, Network Security and Communication Technology (CNSCT '24)*, pages 148–152, New York, NY, USA, 2024. Association for Computing Machinery.
27. A. C. Yao. Protocols for secure computations. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, 1982.
28. C. Zeng, D. He, Q. Feng, X. Yang, and Q. Luo. SecureGPT: A framework for multi-party privacy-preserving transformer inference in GPT. *IEEE Transactions on Information Forensics and Security*, 19:9480–9493, 2024.
29. Fei Zheng, Chaochao Chen, Xiaolin Zheng, et al. Towards secure and practical machine learning via secret sharing and random permutation. *Knowledge-Based Systems*, 245:108609, 2022.
30. Yu Zheng, Qizhi Zhang, Sherman S. M. Chow, Yuxiang Peng, Sijun Tan, Lichun Li, and Shan Yin. Secure softmax/sigmoid for machine-learning computation. In *Proceedings of the 39th Annual Computer Security Applications Conference*, ACSAC '23, page 463–476, 2023.
31. Li Zhengyi, Yang Kang, Tan Jin, et al. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems*, volume 37, pages 21572–21600, 2024.
32. Lijing Zhou, Qingrui Song, Su Zhang, Ziyu Wang, Xiang-Yu Wang, and Yong Li. Bicoptor 2.0: Addressing challenges in probabilistic truncation for enhanced privacy-preserving machine learning. *ArXiv*, abs/2309.04909, 2024. URL: `https://api.semanticscholar.org/CorpusID:261682275`.